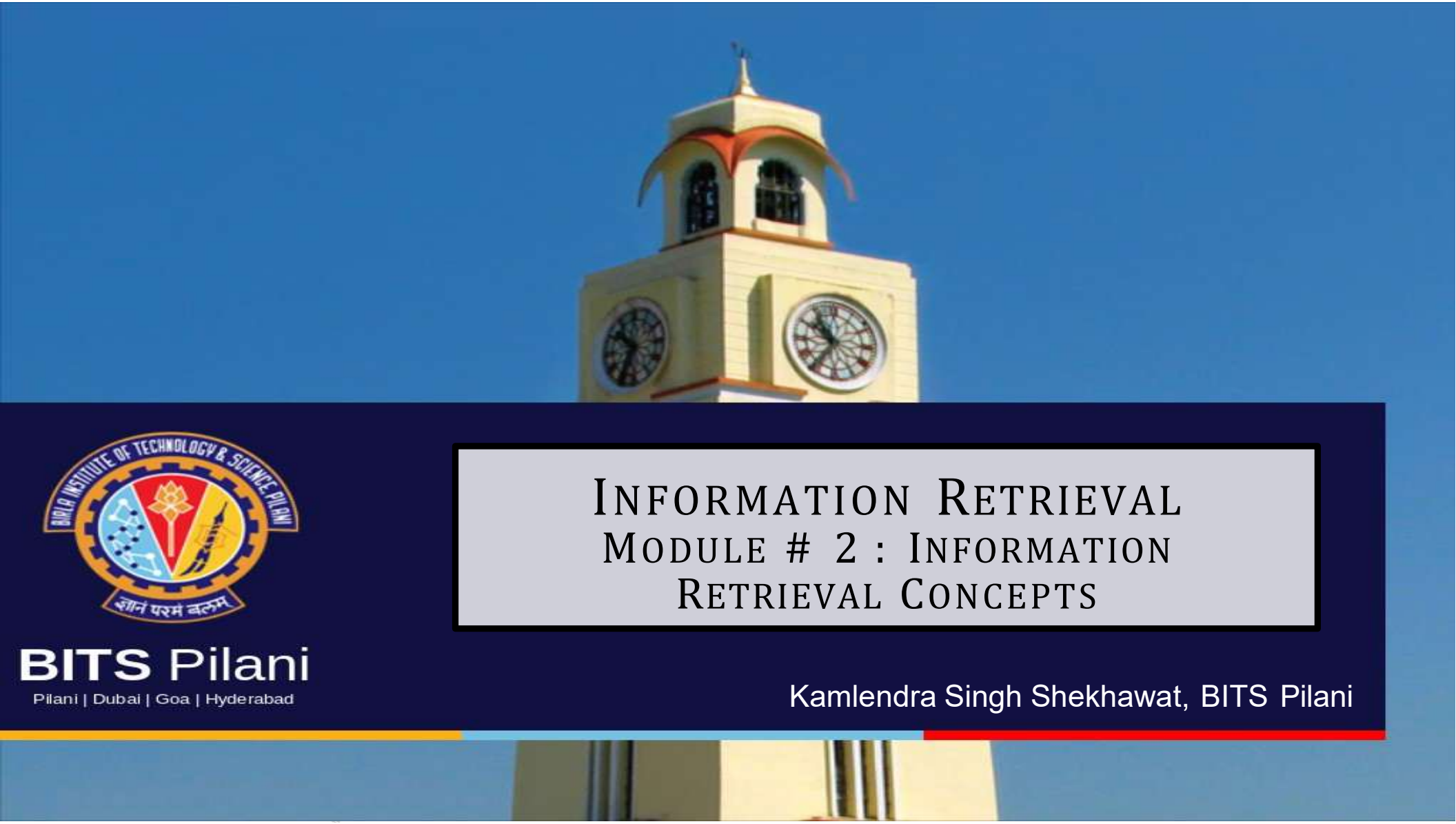


BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Information Retrieval
Module # 2 : Information Retrieval Concepts

Kamlendra Singh Shekhawat, BITS Pilani

The instructor is gratefully acknowledging the authors who made their course materials freely available online.

This deck is prepared by Kamlendra Singh Shekhawat.

Work Integrated Learning Programmes

Table of Contents

1. Module 2
2. Boolean Retrieval
3. Inverted index
4. Processing Boolean Queries
5. Term Vocabulary
6. Phrase Queries

Module Topics....

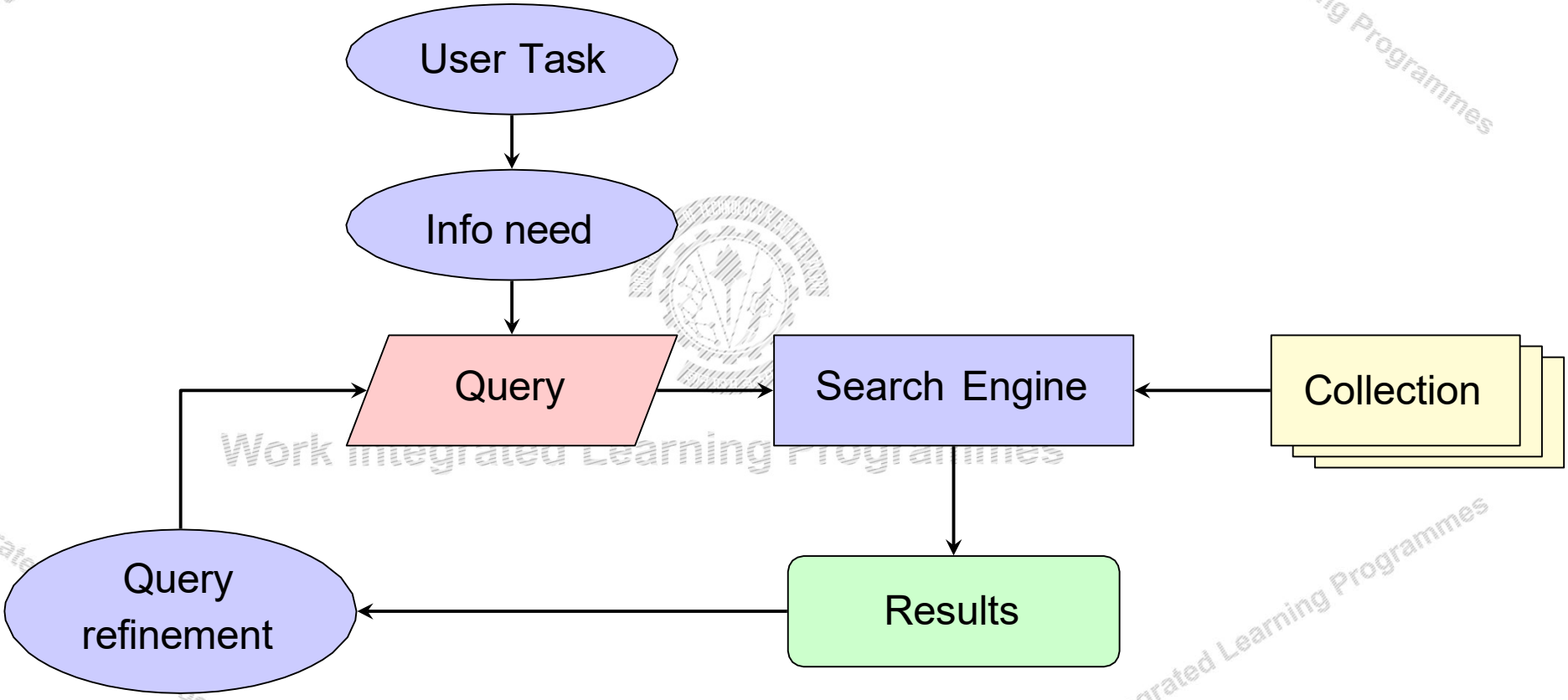
- Boolean Retrieval
- Dictionary and Tolerant Retrieval
- Index Construction and Compression

Work Integrated Learning Programmes

Basic assumptions of Information Retrieval

- Collection:
  - A set of documents.
  - Assume it is static.
- Goal: Retrieve documents with information that is relevant to the user's information need and helps used to complete a task.

Classic Search Model



RELEVANCE

- A document is relevant if it is one that the user perceives as containing information of value with respect to their personal information need.
  - Precision: What fraction of the returned results are relevant to the information need?
  - Recall: What fraction of the relevant documents in the collection were returned by the system?

PRECISION

- Precision in information retrieval refers to the proportion of retrieved documents that are relevant to the user's query.
- Precision measures the accuracy of the retrieved results.

$$Precision = \frac{\text{Total Number of Documents Retrieved}}{\text{Number of Relevant Documents Retrieved}}$$

- High precision indicates that a large portion of the retrieved documents are relevant to the user's query.
- Low precision suggests that many of the retrieved documents are irrelevant.

RECALL

- Recall in information retrieval refers to the proportion of relevant documents that are successfully retrieved by the system out of the total number of relevant documents available in the database.
- Recall measures the accuracy of the retrieved results.

$$Recall = \frac{\text{Total Number of Relevant Documents}}{\text{Number of Relevant Documents Retrieved}}$$

- High recall indicates that the system successfully retrieves a large portion of the relevant documents.
- Low recall suggests that many relevant documents remain undetected or unretrieved.

Precision and Recall - Example

A search engine retrieves documents related to a specific topic. Suppose a user submits a query, and the search engine retrieves a total of 20 documents. Out of these 20 documents, 12 are relevant to the user's query. There are 15 documents which are relevant documents in the database.

$$Precision = \frac{\text{Total Number of Documents Retrieved}}{\text{Number of Relevant Documents Retrieved}} = \frac{12}{20} = 0.6$$

$$Recall = \frac{\text{Total Number of Relevant Documents}}{\text{Number of Relevant Documents Retrieved}} = \frac{12}{15} = 0.8$$

Table of Contents

1. Module 2
2. Boolean Retrieval
3. Inverted index
4. Processing Boolean Queries
5. Term Vocabulary
6. Phrase Queries

Information Retrieval – Example

Problem

Assume that you have the book Shakespeare's Collected Works. Suppose you wanted to determine which plays in that book contain the words **Brutus AND Caesar AND NOT Calpurnia** .

Information Retrieval – Example

Solution 1: Manual read and noting

Start at the beginning and read through all the text, noting for each play whether it contains **Brutus** and **Caesar** and excluding it from consideration if it contains **Calpurnia** .

Information Retrieval – Example

Solution 2: Use Linear scan

For a computer, do linear scan through documents, commonly referred to as grepping through text, (Unix command grep). Grepping through text can be a very effective process, and often allows useful possibilities for wildcard pattern matching through the use of regular expressions.

Information Retrieval – Example

Solution 3: Use Index

To avoid linear scanning the texts for each query, index the documents in advance. Use the index for the search.

Work Integrated Learning Programmes

Information Retrieval – Example

Solution 4: Boolean Retrieval

The Boolean retrieval model is a model for information retrieval in which any query is written in the form of a Boolean expression of terms, that is, in which terms are combined with the operators AND , OR , and NOT .

Boolean Retrieval – Example

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	…
Antony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
…							

1 if play contains word, 0 otherwise

Boolean Retrieval – Example

To answer the query Brutus AND Caesar AND NOT Calpurnia

- S1: take the vectors for Brutus , Caesar and Calpurnia
- S2: complement the last vector (Calpurnia)
- S3: do a bitwise AND
  - 110100 AND
  - 110111 AND
  - 101111 =
  - **100100**
- S4: Answers for this query
  - Antony and Cleopatra, Act III, Scene ii
  - Hamlet, Act III, Scene ii

Boolean Retrieval – Example

To answer the query Brutus OR Caesar

- S1: take the vectors for Brutus , Caesar
- S2: do a bitwise OR
  - 110100 OR
  - 110111
  - **110111**
- S3: Answers for this query
  - Antony and Cleopatra
  - Julius Caesar
  - Hamlet
  - Othello
  - Macbeth

Boolean Retrieval Model

- Record for each document, whether it contains each word out of all the words. In the case of Shakespeare used about 32,000 different words. The result is a binary term-document incidence matrix.
- Terms are the indexed units and they are usually words.
- A term-document incidence matrix. contains
  - a vector for each term, which shows the documents it appears in
  - a vector for each document, showing the terms that occur in it.

Boolean Retrieval Model

- Document means whatever units we have decided to build a retrieval system over.
  - Eg: individual memos or chapters of a book.
  - N = 1 million documents
- Collection or Corpus
  - Group of documents over which we perform retrieval as the (document) collection.
  - Also referred to as a corpus (a body of texts).

Bigger Collections

- Assume Corpus size as N=1 million documents.
- Suppose each document is about 1000 words long (2-3 book pages).
- Assume an average of 6 bytes per word including spaces and punctuation.
- Document collection about 6 GB in size.
- Assume M = 500,000 distinct terms in these documents.
- Term-document matrix is 500K x 1M matrix extremely sparse matrix.
- A much better representation is to record only the things that do occur, that is, the 1 positions.
- This idea is central to inverted index.

Table of Contents

1. Module 2
2. Boolean Retrieval
3. Inverted index
4. Processing Boolean Queries
5. Term Vocabulary
6. Phrase Queries

Inverted Index

- An index always maps back from terms to the parts of a document where they occur.
- Also called inverted file.
- Keep a dictionary of terms (also referred to as a vocabulary or lexicon).
- A list that records each term and documents the term occurs in, is called a postings list (or inverted list).
- Each item in the list, records that a term appeared in a document and the positions in the document, is called a posting.
- All the postings lists taken together are referred to as the postings.
- The dictionary is sorted alphabetically and each postings list is sorted by document ID.

Inverted Index

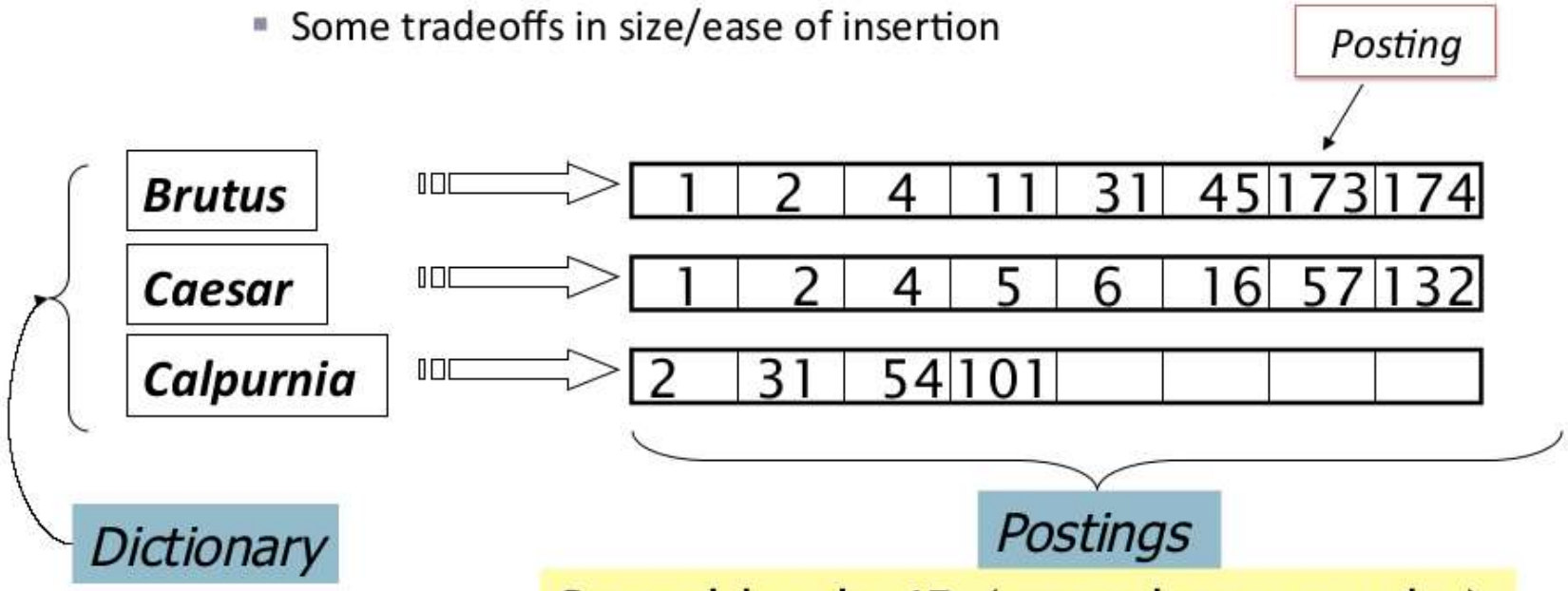
Brutus	→	1	2	4	11	31	45	173	174
Caesar	→	1	2	4	5	6	16	57	132	...
Calpurnia	→	2	31	54	101				
⋮									

Dictionary — Postings

What happens if the word Caesar is added to document 14?

Inverted Index

- We need variable-size postings lists
  - On disk, a continuous run of postings is normal and best
  - In memory, can use linked lists or variable length arrays
    - Some tradeoffs in size/ease of insertion

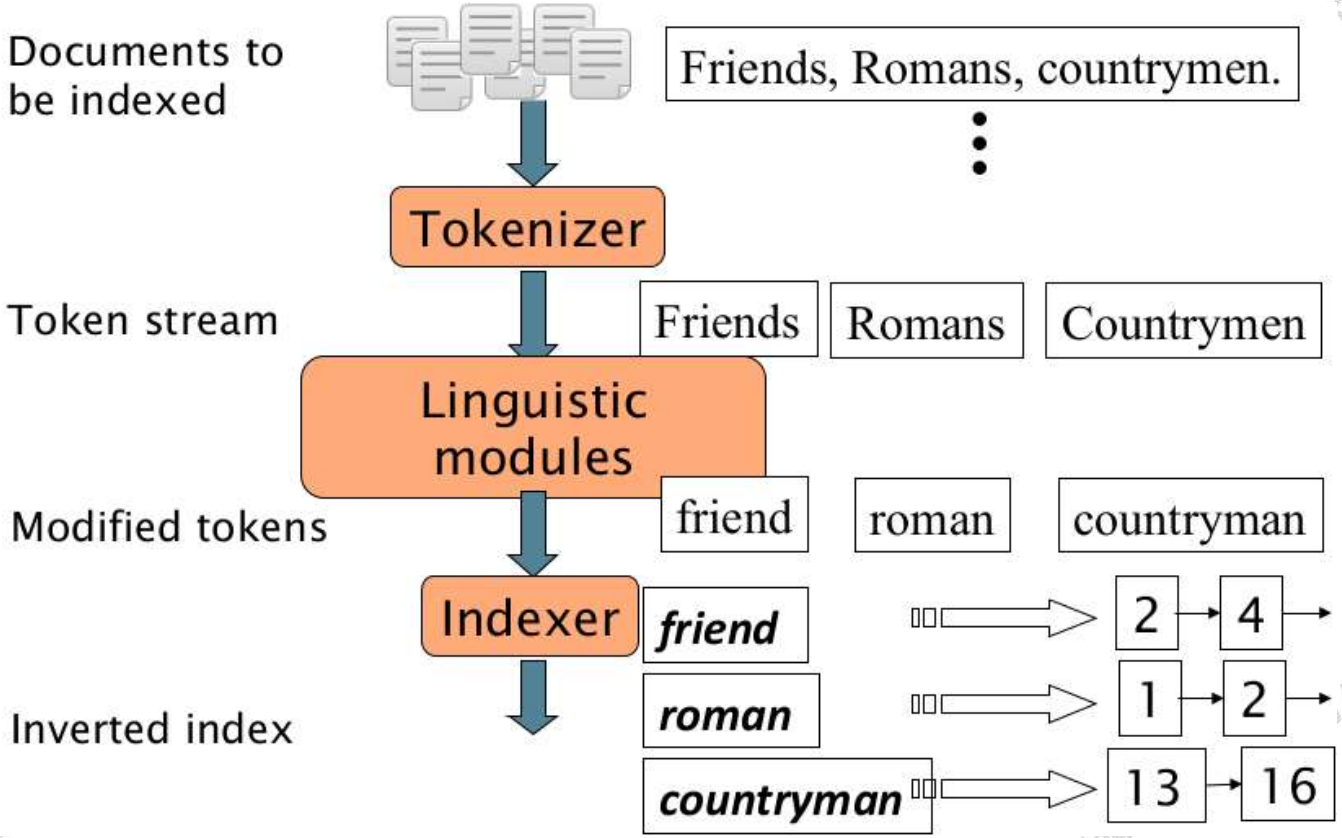


Sorted by docID (more later on why).

Build an Inverted Index

1. Collect the documents to be indexed.
2. Tokenize the text, turning each document into a list of tokens.
3. Do linguistic preprocessing, producing a list of normalized tokens, which are the indexing terms.
4. Index the documents that each term occurs in by creating an inverted index, consisting of a dictionary and postings.

Inverted Index Construction



Initial Stages of Text Processing

- Tokenization
  - Cut character sequence into word tokens
    - Deal with ***"John's"***, ***a state-of-the-art solution***
- Normalization
  - Map text and query term to same form
    - You want ***U.S.A.*** and ***USA*** to match
- Stemming
  - We may wish different forms of a root to match
    - ***authorize***, ***authorization***
- Stop words
  - We may omit very common words (or not)
    - ***the, a, to, of***

Indexer steps: Token Sequence

- Sequence of (Modified token, Document ID) pairs.

Doc 1

I did enact Julius Caesar I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Indexer steps: Sort

- Sort by terms
  - And then docID

Core indexing step

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Indexer steps: Dictionary and Postings

- Multiple term entries in a single document are merged.
- Split into Dictionary and Postings
- Doc. frequency information is added.

Why frequency? Will discuss later.

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
i	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

INDEXER STEPS: STORAGE

term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
i	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1		

Terms and counts

Lists of docIDs

Pointers

IR system implementation

- How do we index efficiently?
- How much storage do we need?

Inverted Index

Doc 1
I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.

Doc 2
So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious:

term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

⟹

term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

⟹

term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
I	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

Table of Contents

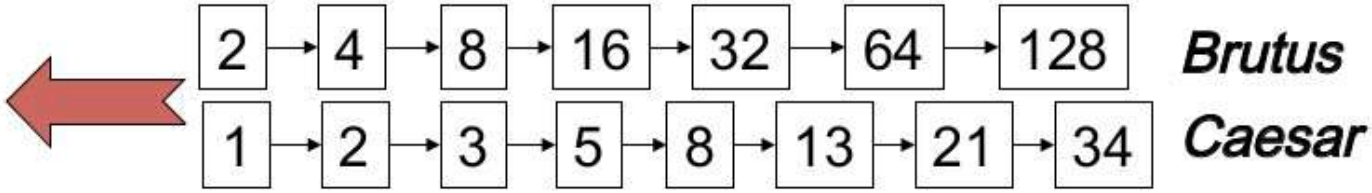
1. Module 2
2. Boolean Retrieval
3. Inverted index
4. Processing Boolean Queries
5. Term Vocabulary
6. Phrase Queries

Query Processing: AND

- Consider processing the query:

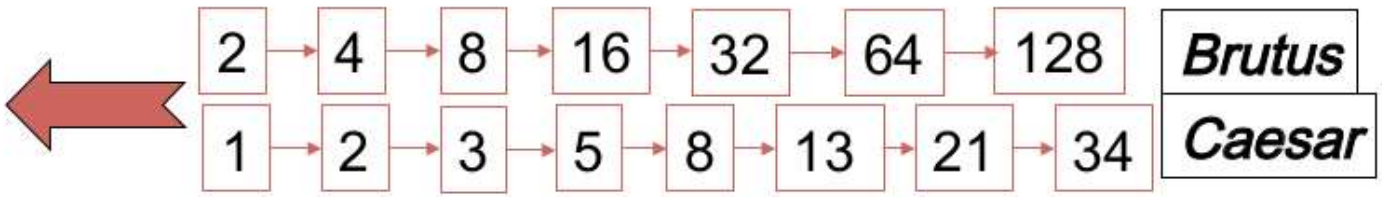
  Brutus *AND* ***Caesar***

  - Locate ***Brutus*** in the Dictionary;
    - Retrieve its postings.
  - Locate ***Caesar*** in the Dictionary;
    - Retrieve its postings.
  - "Merge" the two postings (intersect the document sets):



Query Processing: Merge

- Walk through the two postings simultaneously, in time linear in the total number of postings entries

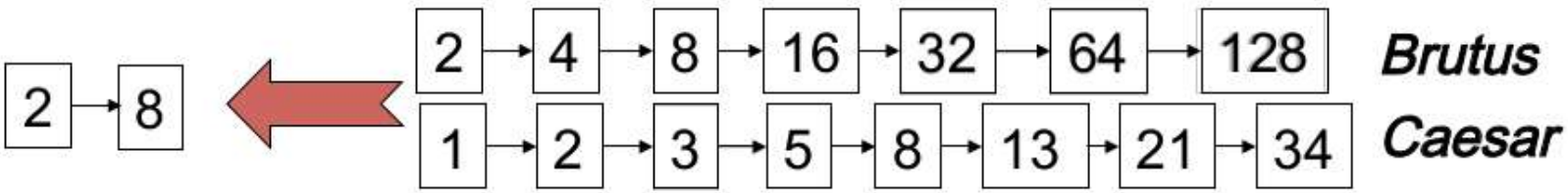


If the list lengths are *x* and *y*, the merge takes O(*x*+*y*) operations.

Crucial: postings sorted by docID.

Query Processing: Merge

- Walk through the two postings simultaneously, in time linear in the total number of postings entries



If the list lengths are *x* and *y*, the merge takes O(*x*+*y*) operations.

Crucial: postings sorted by docID.

Merge Algorithm: Intersect Posting Lists

```
INTERSECT(p1, p2)
  answer ← ⟨ ⟩
  while p1 ≠ NIL and p2 ≠ NIL
  do if docID(p1) = docID(p2)
        then ADD(answer, docID(p1))
             p1 ← next(p1)
             p2 ← next(p2)
        else if docID(p1) < docID(p2)
                then p1 ← next(p1)
                else p2 ← next(p2)
  return answer
```

Exercise

Consider these documents:

- Doc 1 breakthrough drug for schizophrenia
- Doc 2 new schizophrenia drug
- Doc 3 new approach for treatment of schizophrenia
- Doc 4 new hopes for schizophrenia patients

1. Draw the term-document incidence matrix for this document collection.
2. Draw the inverted index representation for this collection.
3. What are the returned results for these queries:
   1. schizophrenia AND drug
   2. schizophrenia OR drug (hint: using AND NOT)

Query Optimization

- Query optimization is the process of selecting how to organize the work of answering a query so that the least total amount of work needs to be done by the system.
- Consideration 1: Order in which postings lists are accessed.
  - Query: AND of *t* terms
    - For each of the t terms, get its postings.
    - Process in order of increasing document frequency.
  - Query: OR of *t* terms
    - Estimate the size of each OR by the sum of its document frequencies = size of each disjunctive term.
    - Process the query in increasing order of the size of each disjunctive term.

EXERCISE

Recommend a query processing order for (tangerine OR trees) AND (marmalade OR skies) AND (kaleidoscope OR eyes) given the following postings list sizes:

Term	Postings size
eyes	213312
kaleidoscope	87009
marmalade	107913
skies	271658
tangerine	46653
trees	316812

EXERCISE

Write out a postings merge algorithm, in the style of INTERSECT(p_1, p_2), for an x OR y query.

Work Integrated Learning Programmes

Table of Contents

1. Module 2
2. Boolean Retrieval
3. Inverted index
4. Processing Boolean Queries
5. Term Vocabulary
6. Phrase Queries

Document Delineation

- Input to Indexing process – Digital documents stored as bytes in a file or on a web server.
- Sequence of characters may be encoded by ASCII or Unicode UTF-8.
- Step 1: Convert byte sequence into a linear sequence of characters.
- Challenge 1: Determine the correct encoding. i.e decode the byte sequence to a character sequence.
- Challenge 2: Choosing a document unit.

Determine Encoding

- Determine the document format
  - Plain text documents requires additional decoding.
  - XML documents character entities, such as &, need to be decoded to give the correct character &
  - Textual part of the document may need to be extracted out of other material that will not be processed. (XML, PDF files)
  - Microsoft Word DOC files
  - Compressed format such as zip files

Choosing Document Unit

- Each file in a folder as a document.
- Regard each email message and each contained attachment as separate documents.
- Decode the zip file and regard each file it contains as a separate document.
- Web software split Powerpoint file into separate HTML pages for each slide or subsection, stored as separate files. In these cases, combine multiple files into a single document.
- For very long documents, the issue of indexing granularity arise. Index each chapter or paragraph as a mini-document.

The person who is deploying IR system must have a good understanding of the document collection, the users, and their information needs and usage patterns.

TOKENIZATION

- Tokenization is the process of chopping character streams into tokens.
- Token is an instance of a sequence of characters in some particular document that are grouped together as a useful semantic unit for processing.
- Type is the class of all tokens containing the same character sequence.
- Term is a (perhaps normalized) type that is included in the IR system's dictionary.
- Example: Identify the terms in `to sleep perchance to dream`
  - Tokens: 5 (each word is a token)
  - Types: 4 (`to` is repeated)
  - Terms: 3 (`to` is removed after normalization)
  - Terms: `sleep, perchance, dream`
- Do the exact same tokenization of document and query words. This guarantees that a sequence of characters in a text will always match the same sequence typed in a query.

Tokenization Steps

- Chop on white-space.
- Throw away certain characters, such as punctuation.
  - Input: Friends, Romans, Countrymen, lend me your ears;
  - Terms: Friends Romans Countrymen lend me your ears
- Split on all non-alphanumeric characters.
  - Input: Mr. O'Neill thinks that the boys' stories about Chile's capital aren't amusing.
  - Term: O Neill Chile s aren t (BAD)
  - Solution; Identify the natural language of the document.

Issues in Tokenization

- Natural Language
  - French: l'ensemble, donne-moi
  - German: compound nouns without spaces (e.g., Computerlinguistik 'computational linguistics') Use compound-splitter.
  - East Asian Languages (Chinese, Japanese, Korean, and Thai), where text is written without any spaces between words. Use word segmentation.
- Hyphens
  - Hewlett-Packard – Hewlett and Packard as two tokens?
  - state-of-the-art
  - co-education

Issues in Tokenization

- White-space
  - San Francisco: one token or two?
  - Los Angeles
  - au fait
- Combination of hyphens and non-separating white-space
  - San Francisco-Los Angeles
  - lowercase, lower-case, lower case
- Different character sequences
  - email addresses (jblack@mail.yahoo.com)
  - Web URLs (http://stuff.big.com/new/specials.html)
  - numeric IP addresses (142.32.48.231)
  - package tracking numbers (1Z9999W99845399981)

Stop Words

- Stop words are extremely common words which would appear to be of little value in helping select documents matching a user need.
- Usually stop words are excluded from the vocabulary entirely.
- Maintain a stop list.
  - Sort the terms by collection frequency (the total number of times each term appears in the document collection)
  - Make a list of the most frequent terms, often hand-filtered for their semantic content relative to the domain of the documents being indexed.
  - This list is the stop list.
- Example: `the, a, and, to, be`
- Web search engines generally do not use stop lists.
- For most modern IR systems, the additional cost of including posting lists for stop words is not that big, neither in terms of index size nor in terms of query processing time.

Issues in Removing Stop Words

- Some special query types are disproportionately affected.
  - King of Denmark
  - President of the United States
  - President AND United States
  - Song titles like Let it be, To be or not to be
- Relational queries:
  - flights to London

NORMALIZATION

- Token normalization is the process of canonicalizing tokens so that matches occur despite superficial differences in the character sequences of the tokens.
- Implicitly create equivalence classes, which are normally named after one member of the set.
  - U.S.A. and USA
  - anti-discriminatory, antidiscriminatory
  - lists of synonyms such as car and automobile

NORMALIZATION

Alternative to creating equivalence classes:

- Maintain lists of synonyms and homonyms between unnormalized tokens.
  - car and automobile
  - color and colour
- Asymmetric expansion
  - Enter: window Search: window, windows
  - Enter: windows Search: Windows, windows, window
  - Enter: Windows Search: Windows
- These approaches are more flexible than equivalence classes because the expansion lists can overlap while not being identical.

NORMALIZATION

- Accents and Diacritics
  - Examples: clich´e and cliche and naive and na¨ıve
  - In English: Remove diacritics.
  - Other languages need special rules.
- Capitalization/case-folding
  - Examples: clich´e and cliche and naive and na¨ıve
  - Approach 1: Reduce all letters to lower case.
  - Approach 2: True casing:
    - convert to lowercase words at the beginning of a sentence and all words occurring in a title that is all uppercase or in which most or all words are capitalized.
    - Mid-sentence capitalized words are left as capitalized
    - e.g., General Motors , Fed vs. fed, SAIL vs.sail

STEMMING

- Stemming refers to a crude heuristic process that chops off the ends of words and removes the derivational affixes.
- To reduce inflectional forms and derivationally related forms of a word to a common base form.
- Reduce terms to their ”roots” before indexing.
- Stemming suggests crude affix chopping.
- Language dependent
  - am, are, is ⟶ be
  - car, cars, car’s, cars’ ⟶ car
  - automate(s), automatic, automation ⟶ automat.
  - the boy’s cars are different colors ⟶ the boy car be different color

Porter's algorithm

- Most common algorithm for stemming English
- Porter's algorithm consists of 5 phases, applied sequentially.
- Within each phase there are various conventions to select rules, such as selecting the rule from each rule group that applies to the longest suffix.
- First phase:

Rule	Example
SSES → SS	caresses → caress
IES → I	ponies → poni
SS → SS	caress → caress
S →	cats → cat

- Later phase: Measure of a word

$(m > 1)EMENT \rightarrow$

replacement → replac

cement → cement

STEMMING

- Lemmatization usually refers to doing things properly with the use of a vocabulary and morphological analysis of words, normally aiming to remove inflectional endings only and to return the base or dictionary form of a word, which is known as the lemma.
- Example: see
  Stemming might return just s
  Lemmatization return either see or saw depending on whether the use of the token was as a verb or a noun.
- It only collapses the different inflectional forms of a lemma.
- Embody transformations that are Language-specific, and Application-specific.
- open source and commercial plug-ins are available

EXERCISE

- Suggest what normalized form should be used for these words:
  - ’Cos
  - Shi’ite
  - cont’d
  - Hawai’i
  - O’Rourke
- The following pairs of words are stemmed to the same form by the Porter stemmer. Which pairs would you argue shouldn’t be conflated. Give your reasoning.
  - abandon/abandonment
  - absorbency/absorbent
  - marketing/markets
  - university/universe
  - volume/volumes

Table of Contents

1. Module 2
2. Boolean Retrieval
3. Inverted index
4. Processing Boolean Queries
5. Term Vocabulary
6. Phrase Queries

Phrase Queries

- Answer queries such as ”Stanford University”, as a phrase.
- The `inventor Stanford Ovshinsky never went to university.` is not a match.
- Double quotes syntax (”stanford university”) for phrase queries, is very easily understood and successfully used by users in search engines.
- More queries are implicit phrase queries.
- For phrase queries, it no longer suffices to store only:
  term: docs

Solution 1: Biword Indexes

- Index every pair of consecutive terms in a document as a phrase.
  - Friends, Romans, Countrymen would generate the biwords: friends romans
    romans countrymen
- Each of the biwords is a dictionary term.
- Two-word phrase query processing is used.

Longer Phrase Queries

- Longer phrases can be processed by breaking them down.
  - Example: The query `stanford university palo alto`
  - broken into the Boolean query on biwords:
  - `stanford university AND university palo AND palo alto`
- Can have false positives.
- Without examining the documents, we cannot verify that the documents matching the above Boolean query do actually contain the original phrase.

Solution 2: Extended Biwords

- Parse the indexed test and perform part-of-speech-tagging.
- Bucket the terms into Noun (N), articles/preposition (X).
- Call any string of terms of the form NX*N an extended biword.
- The extended biword is made a term in the dictionary.
  - Example: renegotiation of the constitution – N X X N
- Query processing
  - parse it into N's and X's
  - segment the query into extended biwords,
  - Look up in the index.

Issues of Biword Indexes

- False positives
- Index blow up due to bigger dictionary
- Biword indexes are the standard solution, but can be part of a compound strategy.

Work Integrated Learning Programmes

Solution 3: Positional Indexes

- In the postings, store for each term the positions in which tokens appear.
- Syntax

```
term, document frequency;
doc1:  position 1, position 2, ...  ;
doc2:  position 1, position 2, ...  ;
```

- Example

```
<be:   993427;
1:   7, 18, 33, 72, 86, 231;
2:   3, 149;
4:   17, 191, 291, 430, 434;
5:   363, 367, ...>
```

- For phrase queries, use merge algorithm recursively at the document level.

Processing a Phrase Query

- Extract the inverted index entries for each distinct term.
- Look for documents that contain both terms.
- Look for places in the lists where there is an occurrence of term 2 with a token index one higher than a position of term 1.
- Look for another occurrence of each word with token index higher than the first occurrence.

Processing a Phrase Query

- Example: to be
  - to: 993427:
    2, 5: 1, 17, 74, 222, 255;
    4, 5: 8, 16, 190, 429, 433;
    7, 3: 13, 23, 191;
  - be, 178239:
    1, 2: 17, 25 ;
    4, 5: 17, 191, 291, 430, 434 ;
    5, 3: 14, 19, 101;
  - to: 429, 433 and be: 430, 434

Proximity Queries

- Same general method is applied for within k word proximity searches.
- /k means within k words of (on either side)
- Example
  `employment /3 place`
- Positional indexes can be used for such queries; biword indexes cannot.

Positional Index Size

- Positional index expands required postings storage significantly, even if indices can be compressed.
- Posting needs an entry for each occurrence of a term.
- The index size depends on the average document size.
  - Average web page has less than 1000 terms.
  - Documents like SEC stock filings, books, and epic poems have 100,000 terms.
- Consider a term with frequency 1 in 1000 (0.1%) terms on average.

Document size	Postings	Positional posting
1000	1	1
100,000	1	100

- Positional index to is 2 to 4 times as large as a non-positional index.
- Positional index size 25-50% of volume of original uncompressed documents.

References

1. Chapter 1 to 5
   C. D. Manning, P. Raghavan and H. Schutze. Introduction to Information Retrieval. 2008.
   http://nlp.stanford.edu/IR-book/

Thank You!